

Corresponding Questions while Exploiting any Unknown SQL Injection Vulnerable Web Application (e.g., DVWA)

Step 1: Identify SQL Injection Vulnerability

Question: What input can you provide to test if the web application is vulnerable to SQL injection?

Expected Answer:

Step 2: Comment Out the Rest of the Query

Question: How can you use SQL comments to ignore the rest of the SQL statement during an injection?

Expected Answer:

Step 3: Display Database Version

Question: What SQL injection payload can be used to display the current database version?

Expected Answer:

Step 4: Display Database Name

Question: What SQL injection payload retrieves the current database name?

Expected Answer:

Step 5: Display Database Version and Database Name Together

Expected Answer:

Step 6: Display Database User and Database Name Together

Expected Answer:

Step 7: Display All Tables in information_schema

The Information Schema stores information about tables, columns, and all the other databases maintained by MySQL.

Question: Write an SQL injection payload to list all tables in the current database.

Expected Answer:

Step 8: Display All User Tables in information_schema

Question: How can you display only the user-related tables in the database?

Expected Answer:

Step 9: Display All Column Fields in the User Table

Question: What payload would you use to display all column names in the `users` table?

Expected Answer:

Step 10: Display Column Field Contents

Question: Provide an SQL injection payload to retrieve the data (like usernames and passwords) from the `users` table.

Expected Answer: